

实验二 线性表实验

顺序表练习一

【问题描述】

题目要求构造一个最大长度为 n ($n \leq 1024$) 的顺序表, 每个元素保存唯一的整型键值 key, 并完成表 10-7 中给出的线性表基本操作, 包括初始化、输出、按位置取值、按位置删除、按键值删除、头插、尾插、指定位置插入以及按键值搜索。实际代码中定义 SeqList 结构体, 使用 `Vec<i32>` 保存元素, 并用 `max_len` 限制顺序表容量。

主程序从 `inputfile.txt` 中读取数据, 第一个整数表示最大长度, 第二个整数表示初始元素个数, 后续整数为初始顺序表元素。程序随后演示查找、插入、删除等操作, 并输出每一步的结果。

【基本思路】

存储结构上, 顺序表采用连续存储思想, Rust 中使用 `Vec<i32>` 模拟数组空间。`new(max_len)` 完成初始化, `output()` 返回当前表内容, `get(pos)` 按 1 开始的位置取元素, `insert(pos, key)` 在指定位置插入, `delete_at(pos)` 删除指定位置元素, `delete_key(key)` 先搜索再删除, `search(key)` 返回键值所在位置。

算法思想与顺序表定义一致: 插入时先判断位置是否合法、容量是否已满、键值是否重复; 删除时先判断位置或键值是否存在, 再移动后续元素。由于 `Vec` 的 `insert` 和 `remove` 会自动完成元素移动, 代码重点放在边界检查和键值唯一性维护上。

```
初始化 SeqList(max_len)
for key in 初始输入元素
    add_tail(key)
输出线性表
get(2) 取得第 2 个元素
search(key) 查找指定键值
add_head(100), insert(2,150), add_tail(200)
delete_at(2), delete_key(200)
输出删除后的线性表
```

函数调用关系可表示为: `main() -> 读取文件数据 -> SeqList::new() -> add_tail()/get()/search()/insert()/delete_at()/delete_key() -> 输出结果。`

【实验结果及分析】

1. 实验结果:

1.1 实验过程描述

1) 本部分采用 Rust 语言开发, 代码组织为 Cargo 工程, 核心顺序表结构和操作函

数位于 `src/lib.rs`，演示程序位于 `src/main.rs`，测试代码位于 `tests/test_fn.rs`。
`inputfile.txt` 示例输入为最大长度 10、初始元素个数 4，初始元素为 10、20、30、40。

- 2) 测试数据重点覆盖表 10-7 的基本操作流程，同时验证非法位置、重复键值和容量满时的拒绝行为，保证顺序表不会产生越界访问或破坏键值唯一性。

编号	测试数据	预期结果 / 实际分析
1	max_len=10；初始表 [10, 20, 30, 40]	可正确输出线性表，get(2)=20，search(10)=1。
2	依次头插 100、位置 2 插入 150、尾插 200	插入后为 [100,150,10,20,30,40,200]，位置移动正确。
3	删除第 2 个元素，再删除键值 200	删除后为 [100,10,20,30,40]，按位置和按键值删除均正确。
4	容量为 2 的表中插入重复键值或非法位置	操作返回 false，原表保持 [1,2]，边界处理正确。

1.2 测试结果

运行 `cargo test` 后，本题 2 个单元测试全部通过；运行 `cargo run` 后，程序按 `inputfile.txt` 输出线性表、查找结果、插入结果和删除结果，与预期一致。

2. 结果及方案分析

2.1 本实验的重点及难点

本实验的重点在于把顺序表的抽象操作完整映射为可调用方法，并统一采用 1 开始的位置编号；难点在于插入和删除时要同时处理位置边界、容量限制和键值唯一性，否则容易出现越界或重复元素。

2.2 本实验设计的优点及可以改进的方向

本实验设计的优点是：结构清晰，接口与表 10-7 一一对应，测试能够覆盖正常流程和异常流程。可以改进的方向包括：为错误情况返回更具体的枚举类型，而不是只返回 `bool` 或 `Option`；进一步把输入命令化，使 `main` 能根据文件内容执行任意操作序列。

3. 还有哪些实现方法？

还可以使用定长数组配合 `length` 字段实现更接近教材定义的顺序表；也可以使用泛型 `SeqList<T>` 支持不同类型的键值。

顺序表练习二

【问题描述】

题目要求编写函数，将一个顺序表 A 分拆成两个顺序表 B 和 C，其中 A 有 n 个元素且元素均不为 0；A 中大于 0 的元素存入 B，小于 0 的元素存入 C。实际代码实现为 split_positive_negative(a: &[i32]) -> (Vec<i32>, Vec<i32>)。

主程序从 inputfile.txt 中读取整数，第一个整数表示 n，后续 n 个整数表示顺序表 A 的元素。程序调用分拆函数后分别输出 A、B、C 三个顺序表。

【基本思路】

存储结构上，输入顺序表 A 用切片 &[i32] 表示，输出顺序表 B 和 C 使用 Vec<i32> 动态数组保存。函数只读取 A，不修改原顺序表，因此传入不可变切片即可。

算法思想是顺序扫描 A 中每个元素：若当前元素大于 0，则追加到 B；若小于 0，则追加到 C。由于题目保证元素不为 0，代码仍然保留了对 0 的忽略处理，使函数在异常输入下也保持稳定。算法时间复杂度为 O(n)，额外空间复杂度为 O(n)。

```
split_positive_negative(A)
  B <- 空表
  C <- 空表
  for item in A
    if item > 0
      B.push(item)
    else if item < 0
      C.push(item)
  return B, C
```

函数调用关系可表示为：main() -> 读取并校验输入 -> split_positive_negative() -> 输出顺序表 A、B、C。

【实验结果及分析】

- 1. 实验结果:
 - 1.1 实验过程描述
 - 3) 本部分代码仍采用 Cargo 工程组织，核心函数位于 src/lib.rs。示例输入为 n=6，A=[3,-1,8,-6,5,-2]，按题意可分拆得到 B=[3,8,5]，C=[-1,-6,-2]。
 - 4) 测试数据既验证正负混合情况，也验证全部为正数或全部为负数时能否正确生成空的另一侧顺序表。

编号	测试数据	预期结果 / 实际分析
1	A=[3,-1,8,-6,5,-2]	B=[3,8,5], C=[-1,-6,-2], 与 inputfile 示例一致。
2	A=[1,2,3]	B=[1,2,3], C=[], 验证全部为正数的情况。

3	A=[-1,-2]	B=[], C=[-1,-2], 验证全部为负数的情况。
4	A 中元素按原顺序交错出现	B 和 C 均保持原表中的相对次序, 稳定分拆。

1.2 测试结果

运行 cargo test 后, 本题 2 个单元测试全部通过; 运行 cargo run 后, 输出 B=[3,8,5]、C=[-1,-6,-2], 与预期一致。

2. 结果及方案分析

2.1 本实验的重点及难点

本实验的重点在于顺序扫描和分类追加, 难点在于分拆后仍要保持元素在原顺序表中的相对次序, 不能为了分类而改变每一类内部的顺序。

2.2 本实验设计的优点及可以改进的方向

本实验设计的优点是: 函数接口简单, 职责单一, 不修改原顺序表。可以改进的方向包括: 当输入中出现 0 时给出提示; 或者把分拆条件抽象为谓词函数, 使同一框架支持更多分类规则。

4. 还有哪些实现方法?

还可以先统计正数和负数个数, 再为 B 和 C 预分配容量, 以减少动态扩容次数; 也可以原地重排 A, 但这样会改变原顺序表, 不如当前方案直观。

顺序表练习三

【问题描述】

题目要求编写函数, 用不多于 $3n/2$ 的平均比较次数, 在顺序表 A 中找出最大值和最小值元素。实际代码实现为 find_min_max(a: &[i32]) -> Option<MinMax>, 其中 MinMax 同时记录最小值、最大值和实际比较次数。

主程序从 inputfile.txt 中读取数据, 第一个整数为 n, 后续 n 个整数为顺序表内容。程序输出最小值、最大值和比较次数。

【基本思路】

普通方法分别寻找最大值和最小值需要约 $2n$ 次比较。为了满足题目要求, 代码采用成对比较法: 每次先比较一对元素, 较小者只与当前 min 比较, 较大者只与当前 max 比较, 因此每两个元素只需要 3 次比较。

当 n 为偶数时, 先比较前两个元素得到初始 min 和 max; 当 n 为奇数时, 以第一个元素同时作为 min 和 max。之后每次处理两个元素, 并累计比较次数。该方法在偶数时比较次数为 $1 + 3(n-2)/2$, 奇数时为 $3(n-1)/2$, 均不超过 $3n/2$ 。

```
find_min_max(A)
if A 为空 return None
```

```
if n 为偶数
    比较 A[0] 和 A[1] 初始化 min, max
    index <- 2
else
    min <- A[0], max <- A[0]
    index <- 1
while index + 1 < n
    先比较 A[index] 和 A[index+1]
    小者与 min 比较, 大者与 max 比较
    index <- index + 2
return min, max, comparisons
```

函数调用关系可表示为: `main()` -> 读取顺序表 -> `find_min_max()` -> 输出最小值、最大值和比较次数。

【实验结果及分析】

- 1. 实验结果:
 - 1.1 实验过程描述
 - 5) 示例输入为 $n=6$, 顺序表 $A=[10,-4,6,18,2,0]$ 。程序先用 10 和 -4 初始化 $min=-4$ 、 $max=10$, 再成对处理 (6,18) 和 (2,0), 最终得到最小值 -4、最大值 18、比较次数 7。
 - 6) 测试数据覆盖奇数长度、偶数长度和空表情况, 并显式断言比较次数不超过 $3n/2$ 。

编号	测试数据	预期结果 / 实际分析
1	$A=[10,-4,6,18,2,0]$	$min=-4$, $max=18$, 比较次数 7, 满足 $7 \leq 9$ 。
2	$A=[7,3,9,-2,5]$	$min=-2$, $max=9$, 比较次数 6, 满足 $6 \leq 7$ 。
3	$A=[]$	返回 None, 说明空表无最大值和最小值。
4	偶数长度与奇数长度分别测试	两种初始化方式均能得到正确结果。

1.2 测试结果

运行 `cargo test` 后, 本题 3 个单元测试全部通过; 运行 `cargo run` 后, 程序输出最小值 -4、最大值 18、比较次数 7。

- 2. 结果及方案分析
 - 2.1 本实验的重点及难点

本实验的重点在于理解成对比较法如何减少比较次数; 难点在于 n 为奇数和偶数时初始化方式不同, 同时要准确统计比较次数。

2.2 本实验设计的优点及可以改进的方向

本实验设计的优点是: 不仅返回最大值和最小值, 还返回比较次数, 便于验证算法复杂

度要求。可以改进的方向包括：返回最大值和最小值所在下标；当存在重复最大值或最小值时说明返回策略。

5. 还有哪些实现方法？

还可以使用分治法分别求左右区间的最大值和最小值，再合并结果；也可以直接两次扫描，但比较次数会增加，不符合本题对 $3n/2$ 的要求。

链表练习二

【问题描述】

题目要求构造一个带头结点的单链表，每个结点中保存唯一的字符型键值 `key`，并完成表 10-7 中的基本操作。实际代码定义 `SinglyList`，内部用 `Option<Box<Node>>` 表示链式结构，`Node` 中保存 `key` 和 `next`。

主程序从 `inputfile.txt` 中读取字符序列，依次尾插生成单链表，然后演示输出、按位置取值、搜索、头插、尾插、按位置删除和按键值删除等操作。

【基本思路】

Rust 中单链表使用 `Box<Node>` 表示堆上结点，用 `Option` 表示空指针。虽然代码没有显式保存一个带数据域为空的头结点，但 `SinglyList` 的 `head` 字段承担头指针管理职责，操作接口按带头结点单链表的逻辑处理。

插入时通过可变链接 `link` 逐步移动到目标位置前，最后把新结点的 `next` 指向原来的后继结点；删除时先找到目标位置对应的链接，再取出结点并让前驱直接指向目标结点的后继。搜索和取值均采用顺序遍历。

```
SinglyList::insert(pos, key)
    if pos 非法或 key 已存在 return false
    link <- head
    移动 link 到第 pos 个位置前
    new.next <- link.take()
    link <- new

SinglyList::delete_at(pos)
    if pos 非法 return None
    link <- head
    移动 link 到目标结点
    target <- link.take()
    link <- target.next
    return target.key
```

函数调用关系可表示为：`main()` -> 读取字符 -> `SinglyList::new()` -> `add_tail()/get()/search()/insert()/delete_at()/delete_key()` -> 输出结果。

【实验结果及分析】

1. 实验结果:

1.1 实验过程描述

- 7) 示例输入为 `a b c d`, 程序先构造单链表 `[a,b,c,d]`, 然后头插 `x`、尾插 `z`, 再删除第 2 个元素和键值 `z`。
- 8) 测试数据覆盖表 10-7 的基本操作, 同时验证重复字符键值、非法插入位置和越界删除位置。

编号	测试数据	预期结果 / 实际分析
1	初始字符序列 <code>a b c d</code>	输出 <code>[a,b,c,d]</code> , <code>get(2)=b</code> , <code>search(a)=1</code> 。
2	头插 <code>x</code> , 尾插 <code>z</code>	插入后为 <code>[x,a,b,c,d,z]</code> , 头尾插入正确。
3	删除第 2 个元素, 再删除键值 <code>z</code>	删除后为 <code>[x,b,c,d]</code> , 链指针调整正确。
4	重复键值 <code>a</code> 或非法位置 0、3	操作被拒绝, 链表结构保持稳定。

1.2 测试结果

运行 `cargo test` 后, 本题 2 个单元测试全部通过; 运行 `cargo run` 后, 单链表输出、搜索、插入和删除结果均与预期一致。

2. 结果及方案分析

2.1 本实验的重点及难点

本实验的重点在于用链式存储实现顺序表中类似的逻辑接口; 难点在于 `Rust` 所有权和可变借用约束下, 插入与删除需要正确移动 `Option<Box<Node>>`。

2.2 本实验设计的优点及可以改进的方向

本实验设计的优点是: 节点结构简单, 接口完整, 测试覆盖重复键值和非法位置。可以改进的方向包括: 显式增加一个头结点字段, 使结构更贴近教材表述; 或者实现迭代器, 便于外部遍历。

6. 还有哪些实现方法?

还可以使用 `Vec` 模拟链表结点池, 以索引代替指针。

链表练习三

【问题描述】

题目要求构造一个不带头结点的单循环链表，每个结点保存唯一的整型键值 key，并完成表 10-7 的基本操作。实际代码定义 CircularList, 用 Vec<Node> 作为结点池, 用 head 保存首结点索引, 用 next 索引形成循环。

主程序从 inputfile.txt 中读取整数，构造单循环链表后输出一轮结果和指定次数的循环输出，再演示取值、搜索、插入和删除操作。

【基本思路】

单循环链表的核心特点是尾结点的 next 指向首结点。代码中每个 Node 保存 key、next 和 active，head 为 None 表示空表；当表中只有一个结点时，该结点的 next 指向自己。

插入头部时，需要找到当前尾结点，把尾结点 next 改为新结点，并更新 head；尾插时，新结点 next 指向 head，尾结点 next 指向新结点。删除首结点时，也要同步更新尾结点 next 和 head，删除唯一结点时则把 head 置空。

```
CircularList::insert(pos, key)
if 空表
    新结点 next 指向自己，head 指向新结点
else
    找到插入位置的前驱和后继
    新结点 next 指向后继
    前驱 next 指向新结点
    若 pos 为 1，更新 head

CircularList::circular_output(count)
从 head 开始重复沿 next 访问 count 次
```

函数调用关系可表示为：main() -> 读取整数 -> CircularList::new() -> add_tail()/circular_output()/get()/search()/insert()/delete_at()/delete_key() -> 输出结果。

【实验结果及分析】

- 1. 实验结果:
 - 1.1 实验过程描述
 - 9) 示例输入为 1 2 3 4，程序输出一轮 [1,2,3,4]，循环输出 8 个元素为 [1,2,3,4,1,2,3,4]，能够证明尾结点重新指向首结点。
 - 10) 测试数据重点覆盖循环输出、删除首结点、删除尾结点、空链表和单结点链表，验证循环指针在边界情况下仍然正确。

编号	测试数据	预期结果 / 实际分析
----	------	-------------

1	初始表 [1,2,3,4]	一轮输出 [1,2,3,4], 循环输出 6 个元素为 [1,2,3,4,1,2]。
2	删除键值 1, 再删除第 3 个元素	删除后为 [2,3], 循环输出仍为 [2,3,2,3,2]。
3	空链表 circular_output(3)	返回空数组, 不发生越界访问。
4	单结点链表 [9]	循环输出 [9,9,9,9], 删除后变为空表。

1.2 测试结果

运行 cargo test 后, 本题 2 个单元测试全部通过; 运行 cargo run 后, 程序能够正确输出循环访问结果和删除后的链表。

2. 结果及方案分析

2.1 本实验的重点及难点

本实验的重点在于维护尾结点到首结点的链接; 难点在于头插、删除首结点和删除唯一结点时, head 与尾结点 next 必须同步更新。

2.2 本实验设计的优点及可以改进的方向

本实验设计的优点是: 通过 circular_output(count) 直观验证了循环结构, 边界测试比较完整。可以改进的方向包括: 维护 tail 索引以减少每次头插或尾插时寻找尾结点的成本。

7. 还有哪些实现方法?

还可以使用 Rc<RefCell<Node>> 表示真实指针循环, 但需要处理引用计数循环的问题。当前使用索引结点池能避免所有权复杂度, 更适合实验验证。

【问题描述】

题目要求构造一个带头结点的双向链表，每个结点保存唯一的整型键值 `key`，并完成表 10-7 的基本操作。实际代码定义 `DoublyList`，用下标 0 作为头结点，其他结点保存 `key`、`prev`、`next` 和 `active`。

主程序从 `inputfile.txt` 中读取整数构造双链表，随后正向输出、反向输出，并演示取值、搜索、头插、尾插、删除等操作。

【基本思路】

双向链表的每个结点同时保存前驱和后继。代码中头结点不保存有效 `key`，初始时 `head.prev=head.next=0`，表示空表；插入新结点时，只要同时修改前驱的 `next` 和后继的 `prev` 即可。

正向输出从头结点 `next` 开始直到回到头结点，反向输出从头结点 `prev` 开始逆向遍历。删除结点时，把前驱 `next` 指向后继，把后继 `prev` 指向前驱，并把目标结点标记为 `inactive`。

```
DoublyList::insert(pos, key)
    找到后继 next
    prev <- next.prev
    new.prev <- prev, new.next <- next
    prev.next <- new
    next.prev <- new

DoublyList::delete_at(pos)
    index <- 第 pos 个结点
    prev <- index.prev, next <- index.next
    prev.next <- next
    next.prev <- prev
    返回 index.key
```

函数调用关系可表示为：`main()` -> 读取整数 -> `DoublyList::new()` -> `add_tail()/output()/reverse_output()/get()/search()/insert()/delete_at()/delete_key()` -> 输出结果。

【实验结果及分析】

1. 实验结果：

1.1 实验过程描述

- 11) 示例输入为 1 2 3 4，程序能正向输出 [1,2,3,4]，反向输出 [4,3,2,1]，随后进行插入和删除操作。
- 12) 测试数据覆盖正向遍历、反向遍历、指定位置取值、按键值查找、按位置删除、按键值删除以及非法操作。

编号	测试数据	预期结果 / 实际分析
----	------	-------------

1	初始表 [1,2,3,4]	正向输出 [1,2,3,4], 反向输出 [4,3,2,1]。
2	头插 100, 尾插 200	插入后为 [100,1,2,3,4,200], 前驱后继均正确。
3	删除第 2 个元素, 再删除键值 200	删除后为 [100,2,3,4], 双向链接保持一致。
4	重复键值和非法位置	操作返回 false 或 None, 表结构不改变。

1.2 测试结果

运行 cargo test 后, 本题 2 个单元测试全部通过; 运行 cargo run 后, 正向输出、反向输出、插入和删除结果均与预期一致。

2. 结果及方案分析

2.1 本实验的重点及难点

本实验的重点在于同时维护 prev 和 next 两个方向的链接; 难点在于插入和删除时四个指针关系必须同时更新, 否则正向和反向遍历结果会不一致。

2.2 本实验设计的优点及可以改进的方向

本实验设计的优点是: 用头结点统一了空表、头插和尾插逻辑, 反向输出可以直接验证 prev 链接。可以改进的方向包括: 释放或复用 inactive 结点空间, 避免长期运行时结点池只增不减。

8. 还有哪些实现方法?

还可以使用标准库 LinkedList, 但它不能直接展示底层 prev 和 next 的维护过程; 也可以使用裸指针实现更接近 C 语言的双链表, 但安全性和复杂度都更高。

链表练习五

【问题描述】

题目给出 polynode 类型，其中 exp 表示指数，coef 表示系数，next 表示下一项指针。要求用链式存储结构生成两个多项式 PA 和 PB，求 PA 与 PB 之和，并输出和多项式。实际代码定义 Polynomial、Node 和 Term，Node 中保存指数和系数，并通过 next 串联。

主程序从 inputfile.txt 读取两行多项式数据。每行第一个数表示项数，后续按“指数 系数”的形式给出各项。程序构造 PA、PB 后调用 add，并输出 PA、PB 和和多项式。

【基本思路】

多项式采用链式结构保存非零项。构造多项式时，代码先使用 BTreeMap 按指数合并同类项并自动排序，再生成按指数降序排列的链表；系数绝对值小于 EPS 的项会被舍弃。

多项式加法的本质是合并两个链表中的项：相同指数的系数相加，不同指数的项保留。代码通过把两个多项式的项展开成数组后重新调用 from_terms，统一完成排序、合并和零项删除。输出时按照指数降序格式化，例如 $1.5x^4 - 3x + 7$ 。

```
Polynomial::from_terms(terms)
    map <- 按指数累加系数
    删除系数为 0 的项
    按指数降序生成链表

Polynomial::add(PB)
    terms <- PA 中所有项 + PB 中所有项
    return Polynomial::from_terms(terms)

format()
    按指数降序输出多项式字符串
```

函数调用关系可表示为：main() -> 读取 PA/PB 项 -> Polynomial::from_terms() -> add() -> format() -> 输出和多项式。

【实验结果及分析】

- 1. 实验结果:
 - 1.1 实验过程描述
 - 13) 示例输入中 $PA=2x^3 - 4x + 7$, $PB=1.5x^4 - 2x^3 + x$ 。相加时 $2x^3$ 与 $-2x^3$ 抵消， $-4x$ 与 x 合并为 $-3x$ ，常数项 7 保留，因此和多项式为 $1.5x^4 - 3x + 7$ 。
 - 14) 测试数据覆盖同类项合并、零项删除、指数降序输出、零多项式格式化等情况。

编号	测试数据	预期结果 / 实际分析
1	$PA=2x^3-4x+7$,	和多项式为 $1.5x^4 - 3x + 7$ ，与 inputfile 一致。

	$PB=1.5x^4-2x^3+x$	
2	同一多项式内出现 (2,3.0) 和 (2,-3.0)	两项抵消后被删除，只保留非零项。
3	$PA=x^2$, $PB=-x^2$	结果为空链表，format() 输出 0。
4	指数 4、3、1、0 混合输入	输出时按指数降序排列，格式清晰。

1.2 测试结果

运行 cargo test 后，本题 3 个单元测试全部通过；运行 cargo run 后，输出 PA、PB 和多项式，结果为 $1.5x^4 - 3x + 7$ 。

2. 结果及方案分析

2.1 本实验的重点及难点

本实验的重点在于链式多项式的项合并和有序输出；难点在于同指数项相加后可能变成 0，需要删除该项，同时格式化输出时要正确处理正负号、指数为 0 或 1 的特殊情况。

2.2 本实验设计的优点及可以改进的方向

本实验设计的优点是：构造函数统一处理排序、合并和零项删除，使 add 函数简洁可靠。可以改进的方向包括：直接用双指针在线合并两个有序链表，减少中间 Vec 和 BTreeMap 的使用；增加多项式求值、乘法等扩展操作。

9. 还有哪些实现方法？

还可以先保持两个多项式链表有序，再像归并排序一样逐项相加；这种方式更贴近教材中的链表合并过程。当前实现更强调代码复用和结果正确性。